

# Priority-Based Genetic Algorithm for the Clustered Steiner Tree Problem

**Tuan-Anh Do**

Hanoi University of Science and Technology

**Ha-Bang Ban** (✉ [bangbh@soict.hust.edu.vn](mailto:bangbh@soict.hust.edu.vn))

Hanoi University of Science and Technology

**Minh Tu Le**

Hanoi University of Science and Technology

**Duc Hung Phan**

Hanoi University of Science and Technology

**Thai Ha Nguyen**

Hanoi University of Science and Technology

**Thi Thanh Binh Huynh**

Hanoi University of Science and Technology

---

## Research Article

**Keywords:** Genetic Algorithm, Clustered Steiner Tree Problem

**Posted Date:** October 31st, 2023

**DOI:** <https://doi.org/10.21203/rs.3.rs-3507907/v1>

**License:** © ⓘ This work is licensed under a Creative Commons Attribution 4.0 International License.

[Read Full License](#)

**Additional Declarations:** No competing interests reported.

---

# Priority-Based Genetic Algorithm for the Clustered Steiner Tree Problem

Tuan Anh Do<sup>1</sup>, Ha-Bang Ban<sup>1\*</sup>, Huynh Thi Thanh Binh<sup>1</sup>, Minh Tu Le<sup>1</sup>, Duc Hung Phan<sup>1</sup> and Thai Ha Nguyen<sup>1</sup>

<sup>1</sup>School of Information and Communication Technology, Hanoi University of Science and Technology, Dai Co Viet, Ha Noi, Vietnam.

\*Corresponding author(s). E-mail(s): [bangbh@soict.hust.edu.vn](mailto:bangbh@soict.hust.edu.vn);

Contributing authors: [anhdt@soict.hust.edu.vn](mailto:anhdt@soict.hust.edu.vn);

[binhht@soict.hust.edu.vn](mailto:binhht@soict.hust.edu.vn); [minhtutx@gmail.com](mailto:minhtutx@gmail.com);

[hung.pd214903@sis.hust.edu.vn](mailto:hung.pd214903@sis.hust.edu.vn); [ha.nt224842@sis.hust.edu.vn](mailto:ha.nt224842@sis.hust.edu.vn);

## Abstract

In a complex network connecting many devices, a set of nodes might be partitioned into multiple local clusters with different functions, properties, or communication protocols. Thus, there has been a rise in network design problems with additional constraints regarding the clustering of vertices, one of them being the Clustered Steiner Tree Problem - a variant of the Steiner Tree Problem. In the literature, there have been a few research working on this problem, but either they only solve it in a metric case, or their exploration capability is still limited. Therefore, their results are not actually good in many cases. To overcome the drawbacks, we propose a Priority-Based Genetic Algorithm to solve the Clustered Steiner Tree Problem. The proposed algorithm maintains a balance between exploration and exploitation to prevent the search from local optima. Experiments and comparisons to existing works in non-metric and metric cases are carried out deliberately to prove the remarkable performance of the proposed algorithm.

**Keywords:** Genetic Algorithm, Clustered Steiner Tree Problem

# 1 Introduction

The Internet of Things (IoT) has led to a rapid increase in the number of interconnected devices with advanced features and capabilities, creating massive computer networks. To facilitate the transmission and management process, ensure the scalability of the network architecture, and protect privacy between different domains, many problems related to partitioning nodes into clusters (or domains) have been proposed. This has led to the development of a class of clustered graph problems, which are important for theoretical and practical reasons, as they allow us to find computation paths between nodes within and between clusters. Some classical graph problems have variants such as the [Clustered Traveling Salesman Problem \(CluTSP\)](#) [1], [Minimum Routing Cost Clustered Tree Problem \(CluMRCT\)](#) [2], [Clustered Shortest-Path Tree Problem \(CluSPT\)](#) [3].

By grouping devices with similar properties into one domain for better management and security, [Clustered Steiner Tree Problem \(CluSteiner\)](#) problem was developed to meet the demands of modern network systems. Its output solution is a graph containing all vertices of different clusters, together with some independent nodes, with the lowest cost possible. Therefore, this problem has practical applications in multipoint networks with clusters of different functionalities and protocols. When costs, latency, power consumption, and bandwidth usage are minimized, networks will operate more efficiently. Throughout history, there have been many research related to this problem, such as designing customized wireless networks to minimize the number of intermediate vertices to reduce latency and collision [4], reduced energy consumption for transmitting signals [5], increased data distribution speed and error handling in connecting data centers [6].

Belonging to the NP-hard class, these problems are usually approached by approximate methods such as [Genetic Algorithm \(GA\)](#) [7, 8] and Tabu search [9] for [CluTSP](#) problem, multifactorial evolutionary algorithms [10] and approximation algorithms [11] for [CluMRCT](#) problem. These approaches divide the problem into two stages: (1) finding a subgraph for each cluster and (2) connecting the subgraphs into a full graph. Following that trend, many studies on the [CluSteiner](#) problem also use two-level approaches to find approximate solutions. However, two-level approaches have some drawbacks. Some algorithms are not suitable for sparse or non-metric graphs. Greedy algorithms, which are used in some two-level approaches, may not explore all possible solutions and can get stuck in local optima. Additionally, some two-level algorithms can be computationally expensive.

In this paper, we propose a method using [GA](#) based on the evolutionary principle of nature that only the fittest individuals can survive. Compared with previous works, we use a priority-based encoding method, and a novel algorithm for finding the Steiner tree is developed. Furthermore, we also conduct experiments on our approach and analyze the results to evaluate its efficiency. The main contributions are summarized as follows:

- Propose a representation method for the [CluSteiner](#) problem and a new scheme to find the Steiner tree according to the given order of vertices to improve the solution space exploration and yield more efficient results.
- Devise a prefiltering process for the graph to reduce the complexity of computing output solutions.
- Evaluate the performance of the proposed algorithm by conducting extensive experiments on various datasets and comparing the results to previous state-of-the-art methods.

The rest of the papers are organized as follows: section 2 presents related works. Section 3 provides problem formulation, while section 4 describes our proposed algorithm in detail. Section 5 contains experimental results and comparisons with other algorithms. Finally, section 6 presents some final remarks and future aspects of our work.

## 2 Related Works

[Steiner Tree Problem \(STP\)](#), named after the mathematician Jakob Steiner, is a classical combinatorial problem. In [12], it was proved to be a problem belonging to the NP-Hard class and has been rigorously studied with various proposals. If the number of required vertices is equal to the number of vertices in the original graph, the problem converts to the problem of finding the [Minimum Spanning Tree \(MST\)](#). Therefore, proposals for the [MST](#) problem can produce a feasible solution for the [CluSteiner](#) problem. The authors in [13, 14] have utilized [MST](#) algorithms and prove that their approach is a 2-approximation algorithm, which means that the cost of its solution is at most twice the cost of the optimal solution in the case of metric graph. Two exact algorithms to solve the [MST](#) problem are Prim [15] and Kruskal algorithms[16]. In other studies, B.Y.Wu and C.W.Lee proposed a two-level algorithm to solve the [CluSteiner](#) problem, [Bilevel Minimum Spanning Tree \(BMST\)](#), which finds the minimum spanning tree for each cluster and connects the sub-trees together using a p-approximation algorithm [13]. In the Clustered Selected-internal Steiner Tree Problem (CSISTP) [14], a variant of the [CluSteiner](#) problem with vertex constraints, Y.H.Chen also used a similar two-level approach. The [MST](#) algorithm is suitable for finding sub-trees at the first level of the [CluSteiner](#) problem because the clusters of the input graph only contain destination vertices without intermediate vertices. However, this method can only be applied when the sub-graphs of G on each cluster are connected graphs. Additionally, the  $(2+p)$  approximation ratio is only valid when the graph is metric.

Notably, in the research about multi-domain Steiner tree [17], L.Chen et al. proposed the [Shortest-Path Heuristic \(SPH\)](#), which initialized the tree with a random vertex in the required set of vertices, then it adds the nearest vertex one by one until all required vertices have been added. The algorithm was proved to have an upper bound of  $2 \times (1 - \frac{1}{k})$  ( $k$  being the number of destination vertices) times the optimal solution. Additionally, the results can be improved

by running the algorithm several times since each starting point gives a different solution. Consequently, under the context of non-metric graph, **SPH** is more efficient than **MST** algorithms. In [17], the authors examined another clustering variant of the Steiner tree problem, with inputs being graphs with all vertices clustered. The problem is approached similarly, with both levels finding the Steiner tree using the **SPH** algorithm. This method can be applied to the **CluSteiner** problem by treating all unconnected intermediate vertices as the intermediate vertex set of the cluster when considering each cluster. At this point, the order of visiting the cluster significantly impacts the algorithm's results.

Based on this foundation, Anh et al. proposed a two-level **SPH** algorithm, **Bilevel Shortest Path Heuristics (BSPH)**, and a genetic algorithm based on **SPH**, **Shortest-Path Genetic Algorithm (SPGA)** [18]. The **BSPH** algorithm randomly selects the order of finding sub-trees for each cluster, while the **SPGA** uses permutation encoding to represent the order. Experimental results show that **SPGA** and **BSPH** have overcome the limitations of **BMST** in non-metric space. However, both used the **SPH** algorithm to find the Steiner tree. Following only the shortest path renders **SPH** incapable of exploring all solution spaces. Additionally, optimizing each cluster separately can make it hard for the overall algorithm to find optimal results: the first visited cluster may use up all intermediate vertices, hence making subsequent clusters and inter-cluster connections less feasible, so the total weight of the tree is increased. Therefore, two-level algorithms similar to **BMST** or **BSPH** cannot improve the solution by using more efficient algorithms to find the Steiner tree, not to mention the possibility of longer computation time.

In this paper, we are dedicated to resolving the aforementioned disadvantages of existing algorithms by proposing a **Priority-Based Search (PBS)** algorithm to find the Steiner tree. To the best of our knowledge, **PBS** is historically an effective method for solving this type of problem. Therefore, it can have remarkable exploration capability in finding the clustered Steiner tree, hence maintaining the diversity of the proposed algorithm. As a result, the proposed algorithm obtains better solutions than others in the literature.

### 3 Problem Formulation

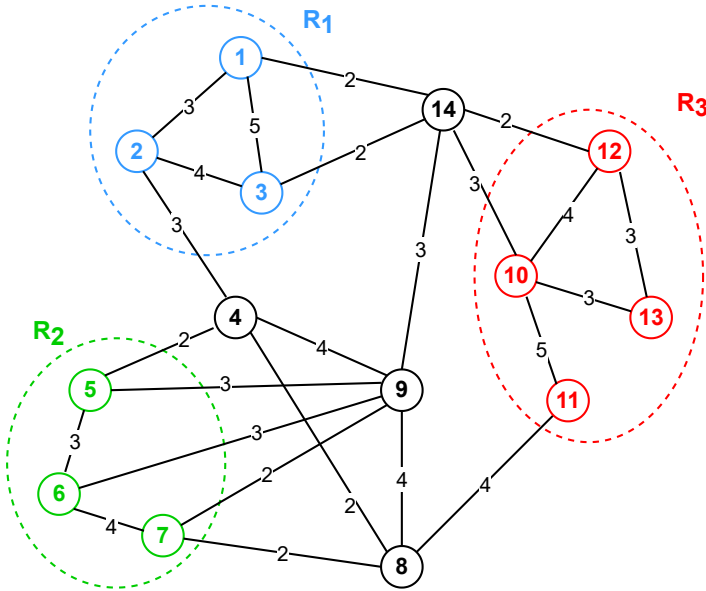
The **CluSteiner** is a variant of the classical **Steiner Tree Problem** in graphs. In the **CluSteiner**, given an undirected weighted graph  $G = (V, E, w)$  and a set of required vertices  $R$ , the objective is to find a minimum weighted acyclic connected subgraph, i.e., a tree, in  $G$  that spans all vertices in  $R$ . The non-required vertices (vertices belong to  $V \setminus R$ ) used as intermediate points in the tree are called Steiner vertices. In addition to the requirements of **Steiner Tree Problem**, the **CluSteiner** also provides a partition  $R' = \{R_1, R_2, \dots, R_k\}$  along with a clustering constraint such that a Steiner tree  $T$  must be a clustered tree for  $R'$ . A Steiner tree  $T$  is a clustered tree for  $R'$  if all the local trees are

mutually disjoint, where a local tree of  $R_i$  in  $T$  is the minimal subtree of  $T$  spanning  $R_i$ .

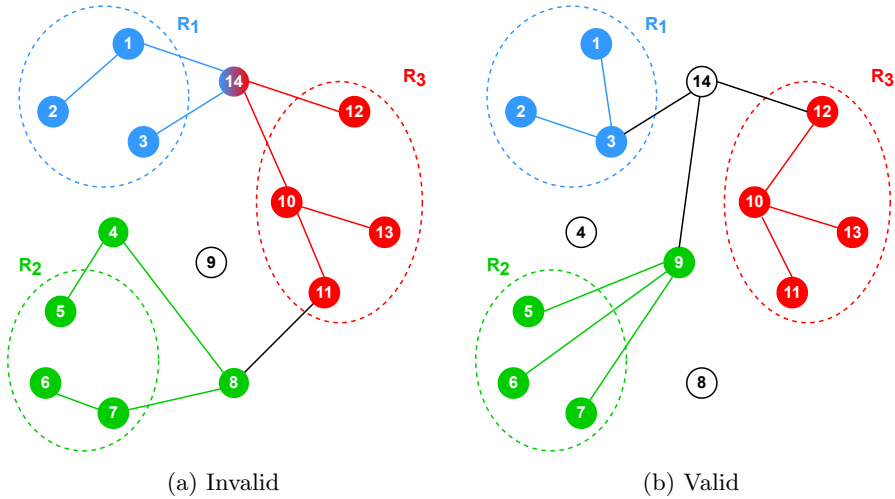
A formal definition for **CluSteiner** is given in table 1.

**Table 1:** Clustered Steiner Tree problem definition

|             |                                                                                                                                                                                                                                                                                                                                                                                                 |
|-------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Input       | <ul style="list-style-type: none"> <li>- A weighted, complete graph <math>G = (V; E; w)</math></li> <li>- A set of required vertices <math>R \subset V</math></li> <li>- A partition <math>R' = \{R_1, R_2, \dots, R_k\}</math> of <math>R</math>; <math>R_i</math> is the <math>i^{th}</math> cluster</li> </ul>                                                                               |
| Output      | A clustered Steiner tree $T = (V_T, E_T)$ for $R'$                                                                                                                                                                                                                                                                                                                                              |
| Objective   | Minimize $\sum_{e \in E_T} w(e)$                                                                                                                                                                                                                                                                                                                                                                |
| Constraints | <ul style="list-style-type: none"> <li>- <math>T</math> is a Steiner tree: <math>R \subset V_T</math></li> <li>- A local tree <math>T_i = (V_{T_i}, E_{T_i})</math> is a Steiner tree of cluster <math>R_i</math>: <math>R_i \subset V_{T_i}</math></li> <li>- All local trees are mutually exclusive: <math>\forall 1 \leq i &lt; j \leq k, V_{T_i} \cap V_{T_j} = \emptyset</math></li> </ul> |



**Fig. 1:** Graph  $G$  with a set of destination vertices partitioned into 3 clusters  $R_1$ ,  $R_2$ , and  $R_3$ .



**Fig. 2:** An example of invalid and valid solution

An example of invalid and valid solution is illustrated in figure 2. In figure 2a, both the minimal local trees of cluster  $R_1$  and cluster  $R_2$  must contain vertex 2 so they are not mutually exclusive, hence the invalidity. Meanwhile, in figure 2b, the graph can be divided into three separate local trees  $\{1, 3\}$ ,  $\{2, 4, 5, 6\}$ , and  $\{8, 9\}$  spanning the three clusters  $R_1$ ,  $R_2$ , and  $R_3$  respectively.

## 4 Proposed Algorithm

Previous sections have introduced the [Clustered Steiner Tree Problem](#). In this section, an approach based on a Genetic Algorithm to solve [CluSteiner](#) is described in detail.

### 4.1 Prefiltering

The process of transforming from the graph  $G$  to  $G'$  is carried out every time an individual is evaluated, thus avoiding repeating the same computational steps between evaluations helps to reduce computation time significantly. This paper analyzes the calculations of the edges' weights in constructing  $G'$  and performs common operations in the prefiltering step. With the sub-tree  $T_i$ , the vertex set  $E_{T_i}$  is partitioned into two parts: the destination vertex set  $R_i$  and the Steiner vertex set  $S_i$ . Set  $S_i$  is determined by the results at the first level and differs between solutions, but  $R_i$  is determined by the input of the problem and remains unchanged. Therefore, the edge weights related to vertices in  $R_i$  are the same between solutions, and can be calculated beforehand to avoid repetition. Considering  $F$  and  $F'$  as the sets of intermediate vertices in  $G$  and  $G'$ , respectively, we can examine the formula for determining the weights of edges in  $G'$  as follows:

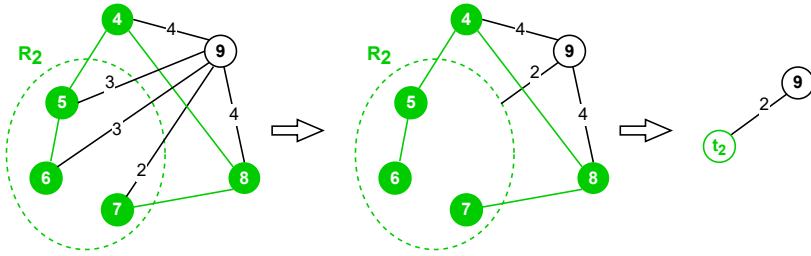
- The weight of the edge between two intermediate vertices  $u, v \in F'$  is equal to the corresponding weight between two vertices in  $G$ .

$$w'(u, v) = w(u, v)$$

- The weight of the edge between an intermediate vertex and the destination vertex: given the definition of the weight between vertex  $u$  and cluster  $R_i$  is  $d(u, R_i) = \min_{v \in R_i} w(u, v)$ , the formula  $w'(u, t_i) = \min_{v \in V_{T_i}} w(u, v)$  is analyzed as follows:

$$\begin{aligned} w'(u, t_i) &= \min_{v \in V_{T_i}} w(u, v) \\ &= \min \left( \min_{v \in R_i} w(u, v), \min_{v \in S_i} w(u, v) \right) \\ &= \min \left( d(u, R_i), \min_{v \in S_i} w(u, v) \right) \end{aligned}$$

If the distance  $d(u, R_i)$  is calculated beforehand, the computational complexity of calculating  $w'(u, t_i)$  decreases by  $O(|R_i|)$ . In Figure 3,  $w'(9, t_2)$  is computed only considering the weights to the cluster  $R_2$  and vertices 4, 8 (set  $S_2$ ) instead of considering all the edges from vertex 9 to 4, 5, 6, 7, 8.



**Fig. 3:** Weight of edge  $(9, t_2)$  with destination vertex  $t_2$  simplified from the tree  $T_2$

- Weight of the edge between two destination vertices: given the definition of weight of the edge between two clusters,  $d(R_i, R_j) = \min_{\substack{u \in R_j \\ v \in R_j}} w(u, v)$ , we have:

$$\begin{aligned}
w'(t_i, t_j) &= \min_{\substack{u \in V_{T_i} \\ v \in V_{T_j}}} w(u, v) \\
&= \min_{u \in V_{T_i}} \left( \min_{v \in R_j} w(u, v), \min_{v \in S_j} w(u, v) \right) \\
&= \min \left( \min_{\substack{u \in R_i \\ v \in R_j}} w(u, v), \min_{\substack{u \in R_i \\ v \in S_j}} w(u, v), \min_{\substack{u \in S_i \\ v \in R_j}} w(u, v), \min_{\substack{u \in S_i \\ v \in S_j}} w(u, v) \right) \\
&= \min \left( d(R_i, R_j), \min_{v \in S_j} d(v, R_i), \min_{u \in S_i} d(u, R_j), \min_{\substack{u \in S_i \\ v \in S_j}} w(u, v) \right)
\end{aligned}$$

Prefiltering step calculates the weights to destination vertices in advance, which helps reduce the computational complexity of the calculation  $w'(t_i, t_j)$  from  $O(|E_{T_i}| \times |E_{T_j}|)$  to  $O(|S_i| \times |S_j|)$ .

The process of constructing the graph  $G'$  for each evaluation has a computational complexity equal to the sum of the computational complexity of calculating the weights:

$$\begin{aligned}
O(\text{compute}(G')) &= O(|F'|^2) + O(|F'| \times \sum_{i=1}^k |S_i|) + O(\sum_{i=1}^{k-1} \sum_{j=i+1}^k |S_i| |S_j|) \\
&< O((|F'| + \sum_{i=1}^k |S_i|)^2) = O(|F|^2)
\end{aligned}$$

## 4.2 Priority-based Search

The **PBS** algorithm is developed based on the **SPH** algorithm and applied to the Steiner tree problem. The tree  $T$  needs to cover a pre-defined set of vertices in a specific order. In each iteration, the algorithm traverses the vertices in the graph and updates the distances from their neighbors to the tree. When the target vertex with the shortest path is determined, **PBS** adds that path to tree  $T$  and starts a new iteration with the next target vertex. The modified **PBS** algorithm is shown in Algorithm 1. Some notations:

- $Q$ : Set of vertices that have not been traversed yet, initially containing all vertices in  $V$ . After traversing a vertex, the algorithm determines the shortest distance from that vertex to tree  $T$ . When tree  $T$  adds a new vertex, the distances from the tree to other vertices may change. Therefore, the algorithm needs to re-traverse the recently added vertex and the vertices that are not part of tree  $T$ . Note that traversing a vertex does not necessarily mean it has been added to tree  $T$ .
- $d(v)$ : Shortest distance from vertex  $v$  to tree  $T$ . Initially,  $d(v) = \infty$  for all  $v \in V$ .
- $p(v)$ : Predecessor vertex of  $v$  on the shortest path to tree  $T$ .

Both the **SPH** and **PBS** algorithms add paths from tree  $T$  to the destination vertices, but they differ in their strategies for selecting the vertices to be added. While **SPH** selects the nearest vertex, **PBS** chooses vertices in a predetermined

**Algorithm 1** Implementation of PBS**Input:**  $G = (V, E, w)$ ,  $R = r_1, r_2, \dots, r_k \subset V$ , the order  $r_1, r_2, \dots, r_k$ **Output:** A Steiner tree  $T = (V_T, E_T) : R \subset V_T$ 


---

```

1:  $V_T \leftarrow \emptyset, E_T \leftarrow \emptyset$ 
2:  $Q \leftarrow V$ 
3:  $\forall v \in V : d(v) \leftarrow \infty$ 
4:  $V_T \leftarrow V_T \cup \{r_1\}, d(r_1) \leftarrow 0$ 
5: for  $i \leftarrow 2$  to  $k$  do
6:   while  $\min_{u \in Q} d(u) < d(r_i)$  do
7:      $u \leftarrow \operatorname{argmin}_{u \in Q} d(u)$ 
8:      $Q \leftarrow Q \setminus \{u\}$ 
9:     for  $\forall (u, v) \in E$  do
10:      if  $d(u) + w(u, v) < d(v)$  then
11:         $d(v) \leftarrow d(u) + w(u, v)$ 
12:         $p(v) \leftarrow u$ 
13:      end if
14:    end for
15:  end while
16:   $u \leftarrow r_i$ 
17:  while  $u \notin V_T$  do
18:     $V_T \leftarrow V_T \cup \{u\}, E_T \leftarrow E_T \cup \{(p(u), u)\}$ 
19:     $d(u) \leftarrow 0, Q \leftarrow Q \cup u$ 
20:     $u \leftarrow p(u)$ 
21:  end while
22: end for
23: return  $T$ 

```

---

order and considers it as part of the problem input. Therefore, though PBS is not as efficient as SPH in finding the Steiner tree, it explores the solution space better and ensures a better overall optimized result in the CluSteiner problem.

### 4.3 Priority-based Genetic Algorithm

#### 4.3.1 Population Initialization

The initial population has a significant impact on the results of algorithm, so it needs to be carefully selected based on the problem, representation and individual evaluation. The evaluation process of individual in Priority-Based Genetic Algorithm (PBGA) uses the PBS algorithm. Although the PBS algorithm has good exploration capabilities in the solution space, it is less efficient than greedy algorithms in constructing new solutions.

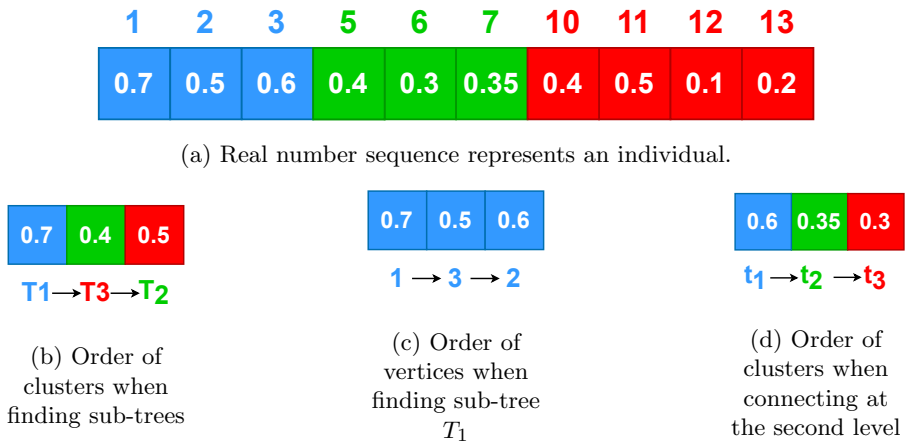
Therefore, the population is randomly initialized, which does not guarantee quality and requires the use of heuristic methods. The proposed algorithm uses BSPH to initialize the population. The BSPH algorithm can generate different solutions by randomly selecting a starting point and the order of searching

for sub-trees. As a result, the initial population ensures both diversity and relatively good quality.

### 4.3.2 Chromosome Representation

The evaluation process is based on two-level approach, in which the order of clusters when searching for sub-trees at the first level must be determined. Additionally, the order of vertices needs to be determined in advance as it is part of the input for the PBS algorithm in finding Steiner trees (presented in 4.2). Therefore, the representation of an individual needs to express the following factors: the order of searching for sub-trees within clusters, the order of destination vertices within each cluster, and the order of searching for destination vertices (extracted from the sub-trees) at the second level.

In the PBGA algorithm, each individual is encoded using a real-valued sequence of length  $|R|$  representing the priority of destination vertices and clusters. At the first level, the highest priority among the vertices in a cluster determines the order of searching for sub-trees. Within a cluster, the priority of vertices determines the traversal order of the PBS algorithm when searching for the sub-tree of that cluster. At the second level, the average priority of vertices within a cluster determines the corresponding order of destination vertices when searching for tree  $T'$ .



**Fig. 4:** An example of representing an individual

Figure 4 illustrates an individual representation for the problem shown in Figure 1. The blue, red, and green colors of the cells in Figure 4a correspond to clusters  $R_1$ ,  $R_2$ , and  $R_3$ , respectively. The numbers above the cells represent the vertices that the cells stand for. The order of clusters when searching for sub-trees is determined by the highest priority within each cluster. In Figure

4b, the order is 0.7, 0.4, 0.5 corresponding to vertex 1 of cluster 1, vertex 5 of cluster 2, and vertex 11 of cluster 3. Therefore, the evaluation process sequentially searches for sub-trees  $T_1$ ,  $T_3$ , and  $T_2$ . For sub-tree  $T_1$ , the PBS algorithm traverses vertices 1,3 and 2 in the priority shown in Figure 4c. The order destination vertices in graph  $G'$  is determined by the average priority of the corresponding cluster. In Figure 4d, the order is 0.6, 0.35 and 0.3. Thus, on  $G'$ , the PBS algorithm searches for the tree  $T'$  in the order of vertices  $t_1, t_2, t_3$ .

### 4.3.3 Chromosome Evaluation

The evaluation process of an individual involves constructing a clustered Steiner tree and calculating its total weight. The solution search process consists of two levels, each level using the PBS algorithm to find a Steiner tree:

- First level: Using the PBS algorithm to find a sub-Steiner tree for each cluster in the order determined by cluster priority.
- Second level: Constructing graph  $G'$  from graph  $G$  and the sub-trees, then finding inter-cluster connections using PBS.

Let  $\alpha(T)$  be the sum of all local trees' costs and  $\beta(T)$  be the sum of all inter-cluster links. The fitness value of the chromosome, i.e., the cost of clustered Steiner tree  $T$ , is:

$$c(T) = \alpha(T) + \beta(T)$$

### 4.3.4 Find the sub-trees within clusters

The order of finding sub-trees is determined by the highest priority of vertices in each cluster. For cluster  $R_{p_i}$  considered at  $i$ th position, the process to find sub-tree  $T_{p_i} = (V_{T_{p_i}}, E_{T_{p_i}})$ , considering the graph  $G_{p_i} = (V_{p_i}, E_{p_i})$  containing:

- The set of vertices  $V_{p_i} = R_{p_i} \cup F_{p_i}$ , with the intermediate vertex set  $F_{p_i}$  consisting of intermediate vertices of  $G$  that have not been used for any previous sub-tree. The set  $F_{p_i}$  is defined as:

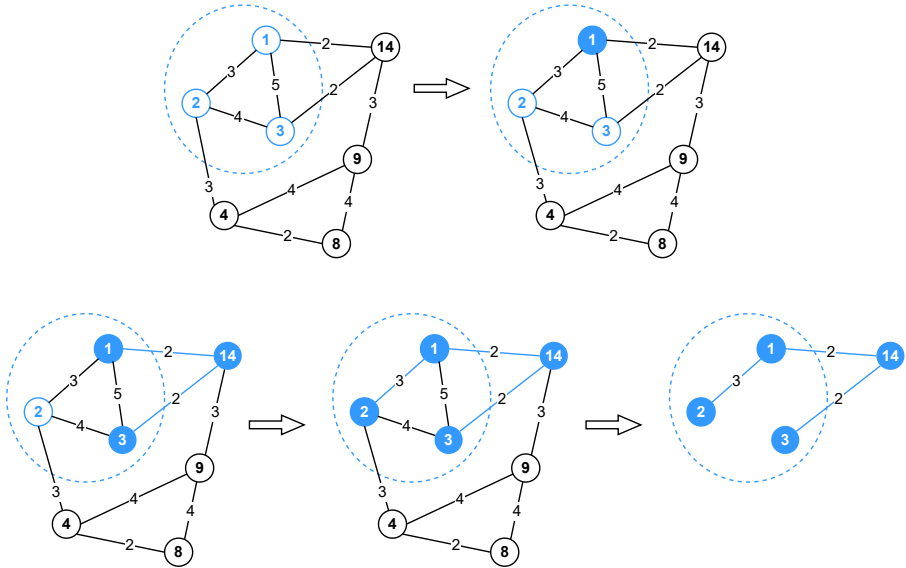
$$F_{p_i} = F \setminus (S_{p_1} \cup S_{p_2} \cup \dots \cup S_{p_{i-1}})$$

where  $F = V \setminus R$  is the intermediate vertex set of  $G$ , and  $S_j$  is the set of Steiner vertices in  $T_j$ .

- The set of edges  $E_{p_i} \subset E$  consists of edges with both endpoints in  $V_{p_i}$ :

$$E_{p_i} = \{(u, v) \in E | u, v \in V_{p_i}\}$$

Using the input of the graph  $G_{p_i}$  and the traversal order of target vertices in  $R_{p_i}$ , the PBS algorithm finds the sub-tree  $T_{p_i}$  for the cluster  $p_i$ . For example, with the individual shown in Figure 4c, the process of finding sub-trees is performed sequentially for clusters  $R_1$ ,  $R_3$ , and  $R_2$ . Among them, the process of finding the sub-tree  $T_1$  is illustrated in Figure 5. Graph  $G_1$  consists of the set of target vertices  $R_1$  and the intermediate vertex set  $F_1$ . Since cluster 1



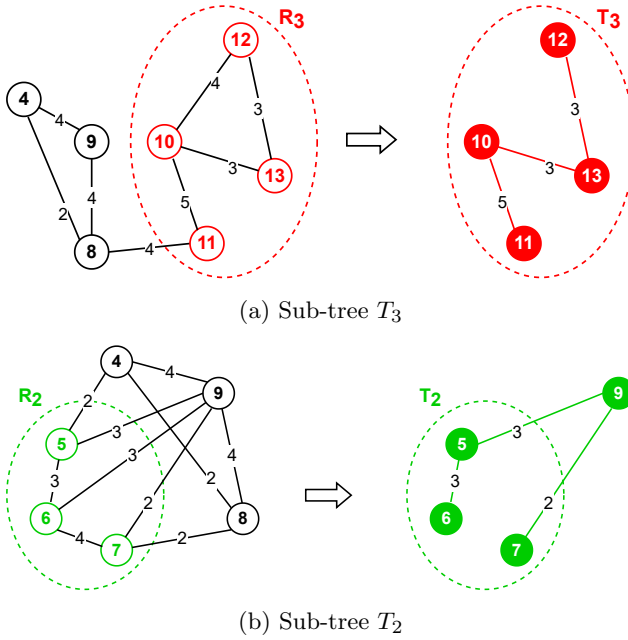
**Fig. 5:** Finding the sub-tree of cluster 1 using **PBS**

is the first cluster to find a sub-tree,  $F_1 = F$ . According to the priority values of the vertices in  $R_1$  in Figure 4c, the traversal order of target vertices is 1, 3, and 2, with decreasing priorities of 0.7, 0.6, and 0.5, respectively. Tree  $T_1$  is initialized with the starting vertex as 1. **PBS** adds vertex 3 to the tree with the shortest path being: 1, 14, 3. Finally, tree  $T_1$  adds vertex 2 with the shortest path being the edge (1, 2). Similarly, the sub-trees  $T_3$  and  $T_2$  found by the **PBS** algorithm are illustrated in Figure 6. Since tree  $T_1$  has the Steiner vertex set  $S_1 = \{14\}$ , the graph  $G_3$  does not contain vertex 14, and the set  $F_3 = F \setminus S_1 = \{4, 8, 9\}$ . Tree  $T_3$  adds the target vertices in the order 11, 10, 13, and 12. Tree  $T_3$  does not have any Steiner vertices, so the graph  $G_2$  has the intermediate vertex set  $F_2 = F_3 = \{4, 8, 9\}$ . The **PBS** algorithm adds the target vertices to  $T_2$  in the order 5, 7, and 6.

After finding the sub-trees, the total weight of the trees is calculated:  $\alpha(T) = \sum_{i=1}^k c(T_i)$ . In the example above,  $\alpha(T) = c(T_1) + c(T_3) + c(T_2) = (3 + 2 + 2) + (5 + 3 + 3) + (3 + 2 + 3) = 26$ .

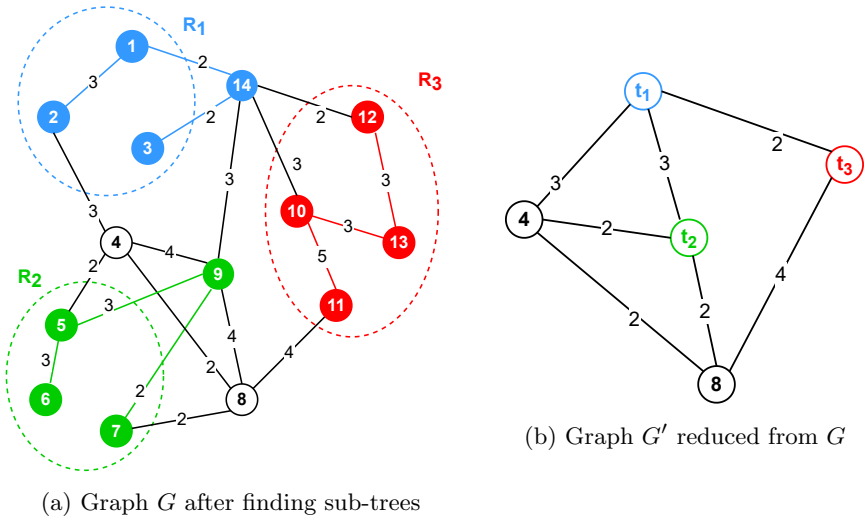
#### 4.3.5 Connecting the sub-trees of each cluster

After finding the sub-trees for each cluster  $T_1, T_2, \dots, T_k$ , the graph  $G$  is reduced to  $G' = (V', E', w')$ , where the target vertex set is  $R' = \{t_i | 1 \leq i \leq k\}$  corresponding to the sub-trees. The second level also utilizes the **PBS** algorithm with the vertex order determined based on the average priority of the clusters to find the tree  $T'$ . The edges of tree  $T'$  represent the connections between the sub-trees, and the cost of inter-cluster connections is denoted as  $\beta(T) = c(T')$ . For the sub-trees  $T_1, T_3, T_2$  obtained in Section 4.3.4, the graph  $G'$  transformed

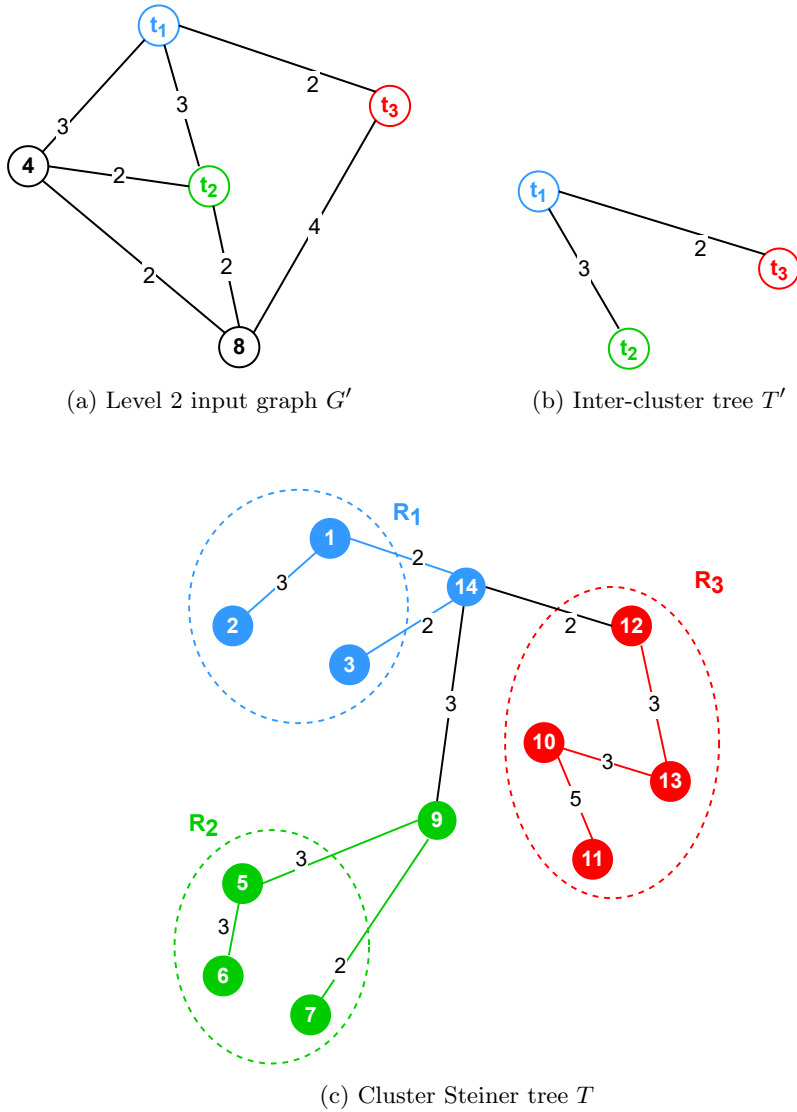


**Fig. 6:** Results of PBS algorithm for input graphs  $G_3$ ,  $G_2$

from  $G$  is illustrated in Figure 7. The PBS algorithm visits the target vertices in the order  $t_1$ ,  $t_2$ ,  $t_3$  as determined in Figure 4d.



**Fig. 7:** Example of graph  $G'$  transformed from  $G$  after finding the sub-trees

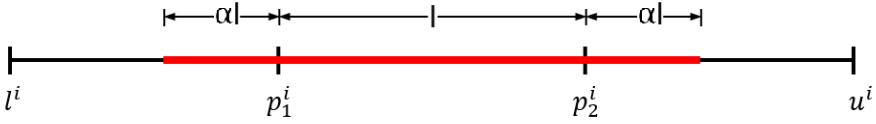
**Fig. 8:** Complete solution of the algorithm

The edges of the tree  $T'$  in Figure 8b are inter-cluster edges connecting the sub-trees,  $\beta(T) = 3 + 2 = 5$ . The tree  $T'$ , along with the sub-trees  $T_1$ ,  $T_3$ ,  $T_2$ , forms the tree  $T$ , which is the complete solution to the problem. The total weight of the tree  $T$  is  $c(T) = 26 + 5 = 31$

## 4.4 Genetic Operator

### 4.4.1 Blended Crossover

In this study, the proposed algorithm utilizes the blend crossover method. Introduced in [19], [Blend crossover \(BLX\)](#) is commonly used with real-valued representations. For a pair of parent individuals  $p_1, p_2 \in R^n$ , the  $i^{th}$  element of the offspring individual  $c$  is randomly generated within the range  $[P_{min} - I \times \alpha, P_{max} + I \times \alpha]$ . Here,  $P_{min}$  is the minimum value between  $p_1^i$  and  $p_2^i$ ,  $P_{max}$  is the maximum value between  $p_1^i$  and  $p_2^i$ , and  $I$  is calculated as  $P_{max} - P_{min}$ . The red area in Figure 9 illustrates the range of values for the offspring's element  $c^i$  generated from the parent individuals  $p_1$  and  $p_2$ .



**Fig. 9:** Range of values for the offspring's element generated from the parent  $p_1$  and  $p_2$  by BLX- $\alpha$

In the case where the value range of the  $i$ -th element is  $[l^i, u^i]$ , [BLX](#) requires normalization of the value  $c^i$  that exceeds the value range: values greater than the range are reassigned to  $u^i$ , and values smaller than the range are reassigned to  $l^i$ .

### 4.4.2 Polynomial Mutation

The mutation operator used in the proposed algorithm is polynomial mutation. First introduced by Deb et al [20, 21], polynomial mutation generates offspring individual  $c$  from parent  $p \in R^n$  by:

- Generating a random value  $u \sim U(0, 1)$
- Using the polynomial distribution to generate parameters for the mutation process:

$$\begin{aligned} \delta_L &= (2u)^{\frac{1}{1+\eta_m}} - 1, & \text{for } u \leq 0.5 \\ \delta_U &= 1 - (2(1-u))^{\frac{1}{1+\eta_m}}, & \text{for } u > 0.5 \end{aligned} \quad (1)$$

where  $\eta_m$  is the mutation parameter.

- The value of element  $i$  of offspring individual  $c$  is calculated using the formula:

$$c^i = \begin{cases} p^i + \delta_L(p^i - l^i), & \text{for } u \leq 0.5 \\ p^i + \delta_U(u^i - p^i), & \text{for } u > 0.5 \end{cases} \quad (2)$$

where  $[l^i, u^i]$  represents the value range of element  $i$ .

The value of offspring individuals generated usually have slight differences from their parents.

### 4.4.3 Elitist Selection

Let  $P_i$  be the population of the current generation  $i$ , and  $C_i$  be the population consisting of offspring generated by crossover and mutation operators. Firstly, all the offspring individuals in  $C_i$  are evaluated. Then, the population of the next generation is selected as the top  $N$  fittest individuals among the combined population of  $P_i$  and  $C_i$ .

## 5 Computational Results

### 5.1 Problem instances

The experiments use the only datasets published in [18]. There are seven datasets comprising 140 instances in total, categorized into two types regarding dimensionality: small instances, each of which has between 30 and 120 vertices, and large instances, each of which has over 260 vertices. Therefore, we have a variety of different classes of instances with varying sizes (different numbers of vertices, edges, and clusters). It allows us to evaluate the efficiency of the proposed algorithm in many scenarios. More details of the datasets are given in table 2.

**Table 2:** Dataset information

| Type         | #NoIns |     | #Vertices | #Clusters | #ReqVertices |
|--------------|--------|-----|-----------|-----------|--------------|
| Type_1_Small | 27     | Max | 105       | 75        | 80           |
|              |        | Min | 51        | 5         | 12           |
| Type_5_Small | 21     | Max | 120       | 10        | 28           |
|              |        | Min | 30        | 5         | 8            |
| Type_6_Small | 37     | Max | 105       | 42        | 51           |
|              |        | Min | 51        | 2         | 12           |
| Type_1_Large | 15     | Max | 242       | 50        | 110          |
|              |        | Min | 262       | 10        | 49           |
| Type_3_Large | 10     | Max | 750       | 25        | 135          |
|              |        | Min | 300       | 6         | 50           |
| Type_5_Large | 15     | Max | 500       | 25        | 96           |
|              |        | Min | 300       | 5         | 49           |
| Type_6_Large | 15     | Max | 442       | 49        | 98           |
|              |        | Min | 262       | 9         | 48           |

\* NoIns: Number of instances, ReqVertices: Required Vertices

### 5.2 Experiment criteria

The quality of the algorithms was assessed by the following criteria:

- **AVG:** The average objective function value over 30 runs.
- **BF:** Best objective function value achieved over 30 runs
- **RPD:** Relative Percentage Difference.
- **PI:** Percentage of Improvement.

Let  $S_{ar}^i$  be the solution produced by algorithm  $a$  in  $r^{th}$  on instance  $i$ . Let  $B^i$  be the best solution among all algorithms, for instance,  $i$ . Then RPD value is calculated using the following equation. The smaller the RPD value, the better the quality of solution found.

$$RPD_{ar}^i = \frac{S_{ar}^i - B^i}{B^i} \times 100\%$$

The Improvement Percentage (PI) is used to signify the improvement of algorithm  $a$  over  $b$ . Let  $AVG_a^i$  and  $AVG_b^i$  be the average value over 30 runs on instance  $i$  of two algorithms  $a$  and  $b$ , respectively. Then the improvement of the algorithm  $a$  over algorithm  $b$  is:

$$PI_{ab}^i = \frac{AVG_b^i - AVG_a^i}{AVG_b^i} \times 100\%$$

### 5.3 Experimental Setting

Each algorithm is executed independently 30 times on each dataset. The computer configuration is Intel(R) Core(TM) i5-3470 CPU 3.2GHz, 16GB RAM. The algorithms are implemented in the Java language. The parameters of SPGA and PBGA are provided by Table 3.

**Table 3:** Parameters of SPGA and PBGA

| Parameter             | Definition                              | Value  |
|-----------------------|-----------------------------------------|--------|
| <i>POPSIZE</i>        | Number of individuals in the population | 100    |
| <i>MAXGENERATIONS</i> | Number of generations                   | 500    |
| <i>MAXEVALUATIONS</i> | Number of evaluations                   | 50,000 |
| $p_m$                 | Mutation rate                           | 0.05   |
| $p_c$                 | Crossover rate                          | 0.9    |
| $n_m$                 | Mutation parameter PM                   | 15     |
| $\alpha$              | BLX crossover parameter                 | 0.3    |
| <i>MAPSIZE</i>        | Size of the SMS map                     | 15     |

To ensure objectivity, the number of evaluations for the solutions of each algorithm must be the same. SPGA and PBGA have the same population size of 100 and the same number of generations of 500, which corresponds to 50,000 evaluations. The BSPH algorithm evaluates only one solution at a time, so it needs to be run 50,000 times (random starting vertex and subtree finding order) to find the best solution. In addition, these 50,000 runs share the preprocessed graph information in 4.1 to ensure fairness in terms of time with SPGA and PBGA.

## 5.4 Experimental Scenario

The performance of proposal algorithm was compared with two other algorithms BSPH and SPGA. There are two experiments as follows:

*Experiment 1:* Conduct non-parametric statistic tests to analyze the results of the algorithms.

*Experiment 2:* Compare results, run time, and convergence trend of difference algorithms.

## 5.5 Experimental results

### 5.5.1 Non-parametric statistics to compare the results of the proposed algorithm and the existing algorithm

Statistical non-parametric tests are used to examine whether significant differences exist in the results of the BSPH, SPGA, and PBGA algorithms. The process of conducting non-parametric statistical analysis consists of two steps:

In the first step, the Friedman, Aligned Friedman, and Quade tests [22] are employed to evaluate the differences among the algorithm results.

In the second step, after confirming the differences among the algorithms, further post-hoc statistical tests are conducted to compare the best-performing algorithm (or the control algorithm) with the other two algorithms.

The results of the Friedman, Aligned Friedman, and Quade tests are presented in Table 4. The p-values for all tests are below the threshold of 0.05, indicating significant statistical differences in the obtained results among the algorithms.

**Table 4:** The results of Friedman, Aligned Friedman and Quade statistical tests( $\alpha = 0.05$ )

| Friedman Value<br>211.444 |                               | Aligned Friedman Value<br>90.666 |                               | Quade Value<br>221.894  |                               |
|---------------------------|-------------------------------|----------------------------------|-------------------------------|-------------------------|-------------------------------|
| Value in $X^2$<br>5.991   | p-value<br>$9.448 * 10^{-11}$ | Value in $X^2$<br>5.991          | p-value<br>$8.546 * 10^{-11}$ | Value in $F_F$<br>3.029 | p-value<br>$1.005 * 10^{-57}$ |

The average rankings of the algorithms are presented in Table 5. The results show that the proposed PBGA algorithm significantly outperforms BSPH and SPGA on all types of statistical tests.

In the second step, since PBGA has the lowest rank, it is chosen as the control algorithm for pairwise comparisons with the other two algorithms using the Holland and Holm statistical methods. The p-values in Table 6 are much smaller than the threshold  $\alpha = 0.05$ , indicating the superiority of PBGA over the other two algorithms.

**Table 5:** The average rankings of the algorithms are computed using the Friedman, Aligned Friedman, and Quade statistical tests.

| Algorithm | Friedman | Friedman Aligned | Quade |
|-----------|----------|------------------|-------|
| BSPH      | 2.425    | 299.279          | 2.636 |
| SPGA      | 2.004    | 202.967          | 1.977 |
| PBGA      | 1.571    | 129.254          | 1.387 |

**Table 6:** z-values and p-values of the Friedman statistical test with the control algorithm PBGA

| $i$ | Algorithm | $z = (R_0 - R_i)/SE$ | $p$       | Holm/Hochberg/Hommel | Holland |
|-----|-----------|----------------------|-----------|----------------------|---------|
| 2   | BSPH      | 7.141                | 9.232E-13 | 0.025                | 0.025   |
| 1   | SPGA      | 3.616                | 2.997E-4  | 0.05                 | 0.05    |

**Table 7:** PI (%) of PBGA compared with BSPH

|       | Dataset      | PI min    | PI average | PI max    | + / $\approx$ / - |
|-------|--------------|-----------|------------|-----------|-------------------|
| Small | Type_1_Small | -0.007416 | 0.362646   | 3.646409  | 6/19/2            |
|       | Type_5_Small | 0.000000  | 0.202814   | 1.323529  | 5/16/0            |
|       | Type_6_Small | -0.040123 | 0.526952   | 10.554090 | 13/22/2           |
| Large | Type_1_Large | -0.129874 | 2.208344   | 6.692581  | 14/0/1            |
|       | Type_3_Large | 0.387083  | 3.330297   | 6.362738  | 10/0/0            |
|       | Type_5_Large | 0.131776  | 2.391222   | 6.159098  | 15/0/0            |
|       | Type_6_Large | -0.159064 | 2.003998   | 4.675739  | 14/0/1            |

### 5.5.2 Detail of comparison among the algorithms **BSPH**, **SPGA** and **PBGA**

This section provides a detailed comparison of the algorithms, including the results, running time, and convergence trend. The results are evaluated based on the criteria of average PI improvement on each dataset and the relative percentage difference (RPD).

The improvement in the average PI results of the PBGA algorithm compared to BSPH and SPGA is presented in Tables 7 and 8.

The BSPH and SPGA algorithms yield similar results on small datasets because they both use SPH to find Steiner trees, which does not fully explore the solution space. On the other hand, PBGA shows significant improvement, especially on the Type\_6.Small dataset.

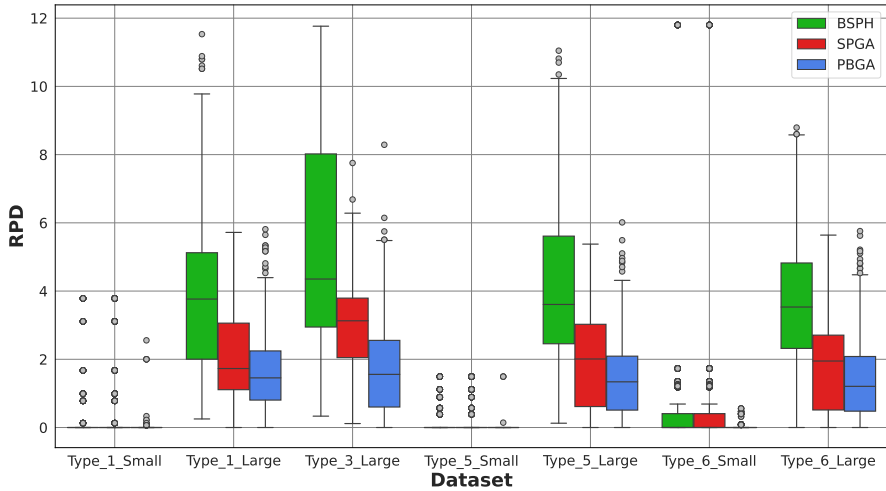
Although PBGA does not completely outperform the other algorithms on large datasets, it performs better on most samples. BSPH produces inferior results compared to PBGA on all samples, except for sample 50gil262 from the Type\_1.Large dataset and 9a280-3x3 from the Type\_6.Large dataset. While SPGA has better results on a few samples, PBGA overall proves to be more

**Table 8:** PI (%) of PBGA compared with SPGA

|       | Dataset      | PI min    | PI average | PI max    | + / $\approx$ / - |
|-------|--------------|-----------|------------|-----------|-------------------|
| Small | Type.1.Small | -0.007416 | 0.362646   | 3.646409  | 6/19/2            |
|       | Type.5.Small | 0.000000  | 0.202814   | 1.323529  | 5/16/0            |
|       | Type.6.Small | -0.040123 | 0.526952   | 10.554090 | 13/22/2           |
| Large | Type.1.Large | -0.677928 | 0.370860   | 1.860520  | 9/0/6             |
|       | Type.3.Large | -0.167542 | 1.129364   | 2.608146  | 9/0/1             |
|       | Type.5.Large | -1.687057 | 0.529224   | 2.713064  | 12/0/3            |
|       | Type.6.Large | -0.639308 | 0.321106   | 1.886703  | 8/0/7             |

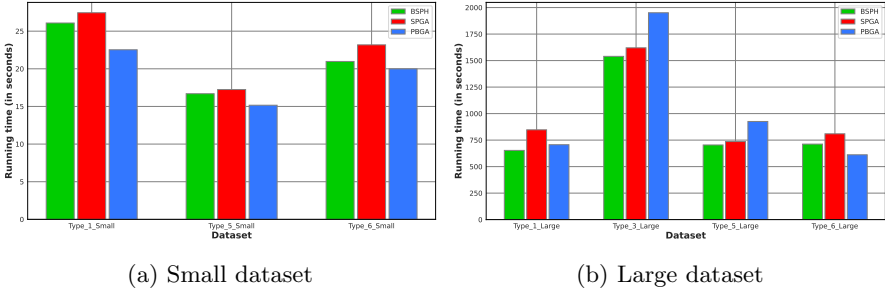
effective. PBGA demonstrates improvement compared to BSPH and SPGA, with an average PI improvement ranging from 0.20 to 3.33.

To evaluate the aspect of finding the best solutions, Figure 10 illustrates the distribution of RPD values for each algorithm.

**Fig. 10:** RPD values of PBGA compared to BSPH and SPGA

PBGA achieves the lowest RPD values, around 2%. Additionally, the consistency of the results across different runs and samples is evident from the short and closely located boxplots, indicating minimal variation.

To evaluate the running time, this study calculates the average running time in seconds of the algorithms on the samples of each dataset and illustrates it in Figure 11. On small datasets, PBGA has the lowest running time, but the difference is not significant. On large datasets, there is a noticeable difference between the datasets. PBGA has the lowest running time on Type.1.Large and Type.6.Large but the highest on Type.3.Large and Type.5.Large.



**Fig. 11:** Average running time of the algorithms on different datasets BSPH, SPGA and PBGA by seconds

### 5.5.3 Convergence trends and time running

As SPGA and PBGA tend to converge quickly within the first few generations on small datasets, this study focused on evaluating the convergence trends of these two algorithms on large datasets. The convergence trend of each dataset is assessed by calculating the average fitness value for each generation over 30 runs. The convergence trends of selected instances from different datasets are illustrated in Figure 12.

To analyze the convergence trend on the dataset, the convergence rate can be evaluated using the normalized average value of the dataset, based on the formula described in [23]. The process of calculating the normalized average value of algorithm  $A$  on dataset  $T$  is as follows:

- Compute the average solution value (based on 30 runs) for each generation on dataset  $i$  of  $T$ . The average solution value of algorithm  $A$  at generation  $t$  on dataset  $i$  is denoted as  $AVG_{At}^i$ .
- Calculate the normalized value for each dataset  $i$ . At generation  $t$ , the normalized value is computed using the following formula:

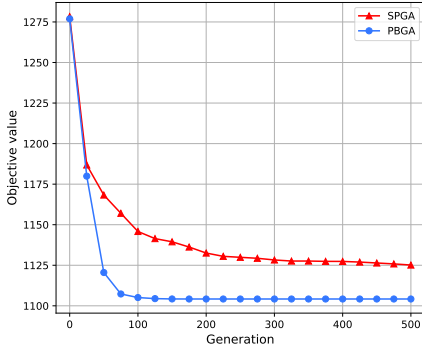
$$s_{At}^i = \frac{AVG_{At}^i - AVG_A^i}{AVG_{A0}^i - AVG_A^i}$$

Here,  $AVG_{A0}^i$  represents the average solution value when initialized, and  $AVG_A^i$  represents the average solution value of algorithm  $A$  after 500 generations. In case there is no improvement compared to the initialization value ( $AVG_{A0}^i = AVG_A^i$ ), we assume  $s_{At}^i = 0$  for all  $t$ .

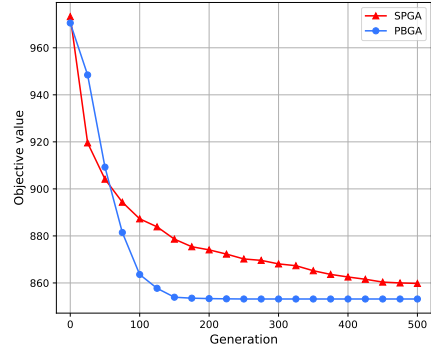
- Calculate the normalized value for the entire dataset  $T$  by taking the average of the normalized values across all dataset instances:

$$s_{At}^T = \frac{1}{|T|} \sum_{i \in T} s_{At}^i$$

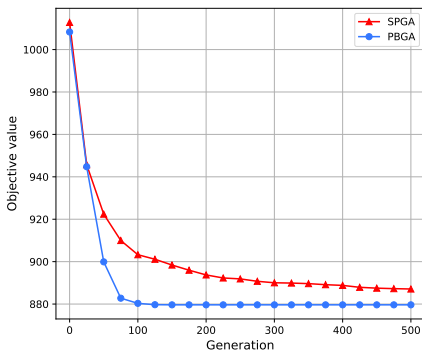
the normalized value belongs to  $[0, 1]$ .



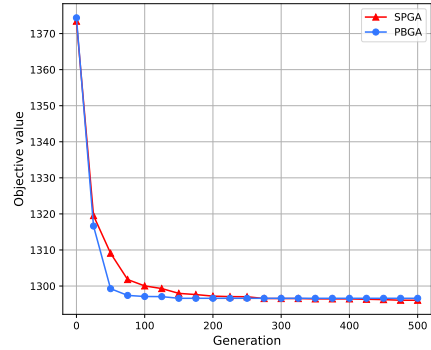
(a) Instance 50pr439 of Type\_1.Large



(b) Instance 25i750 of Type\_3.Large



(c) Instance 25i500-308 of Type\_5.Large



(d) Instance 49lin318-7x7 of Type\_6.Large

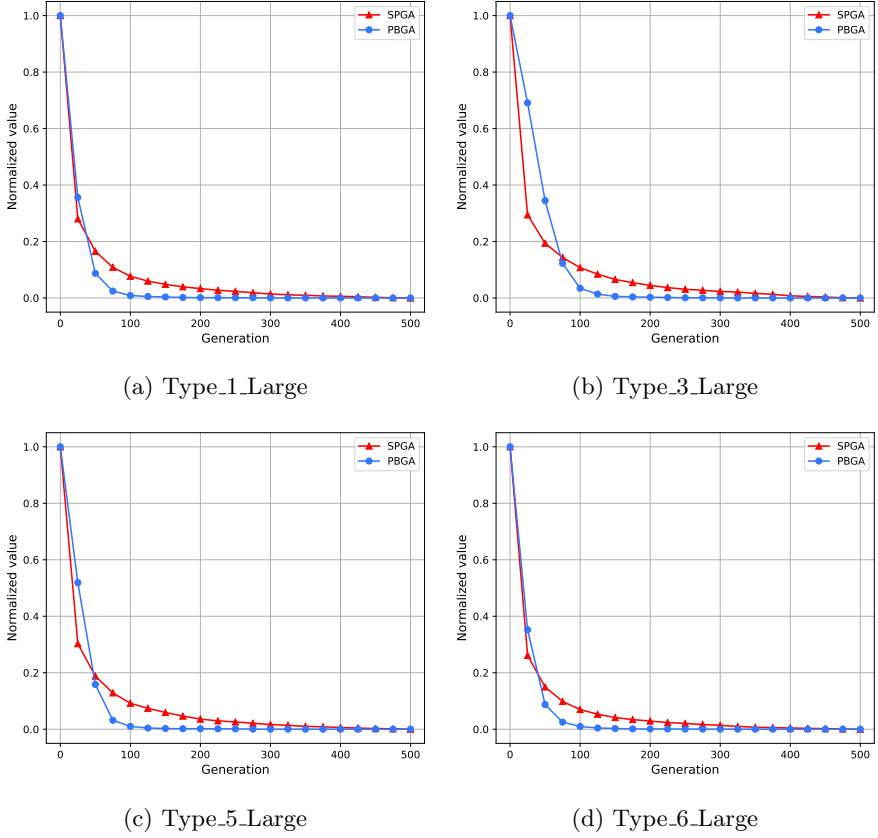
**Fig. 12:** Convergence trends of SPGA and PBGA on selected instances

Using this normalization approach, the convergence trend of SPGA and PBGA algorithms on several large datasets is illustrated in Figure 13.

The comparison between the two algorithms shows that PBGA tends to converge slower than SPGA in the first 50 generations, but it improves rapidly in the next 50 generations. Therefore, maintaining population diversity through tiered selection in PBGA helps ensure the exploration factor, surpassing SPGA in the 50-100 generation range. However, from generation 100 onwards, both algorithms gradually converged, and after generation 200, no significant improvement was observed.

## 5.6 Dicussions

Generally speaking, a good metaheuristic needs to balance between exploration and exploitation capacity. The exploration characteristic means the search



**Fig. 13:** Convergence trends of SPGA and PBGA in many dataset

explores new promising solutions, while exploitation means the search effectively exploits the current solution space. Related to the problem, we found that only one metaheuristic SPGA was proposed to solve the problem in the literature, though it is an interesting problem.

The SPH method in SPGA is mainly based on a greedy approach to finding the Steiner tree. However, it is too greedy and does not support enough diversity to maintain the exploration capacity for SPGA. Therefore, SPGA can get trapped in the local optima. On the other hand, in finding the Steiner tree, the PBS in PBGA provides a promising list of candidate vertices to visit. In each step, we select a vertex from the list. Therefore, it brings randomness and greediness together. The size of the list controls the balance between greediness and randomness. This balance helps PBGA to maintain a diversity of the population. Additionally, the search is prevented from premature convergence in many cases. As a result, PBGA is better than SPGA in terms of solution quality and convergence trends in most cases.

## 6 Conclusion

This paper introduced a two-level **Genetic Algorithm (GA)** using **Priority-Based Search (PBS)** to solve the **CluSteiner** problem with two contributions. First, we propose the **PBS** algorithm to find better Steiner trees maintaining exploration capacity. Second, we introduce the efficiency algorithm based on a genetic algorithm scheme with priority-based encoding. The algorithm has a good balance between exploration and exploitation. Its efficiency, in terms of solution quality, computational time, and convergence trends, was evaluated by extensive experiments. The results show that **PBGA** is superior to other existing algorithms for most cases. The new best solutions can be reached in many cases. However, the running time needs to be improved. That is our aim for future improvement.

**Acknowledgments.** This work is funded by the Ministry of Education and Training of Vietnam for Do Tuan Anh under project code B2023-BKA-05. It was also sponsored by the Office of Naval Research (ONR) Global (ONRG) and the U.S. Army Combat Capabilities Development Command Indo-Pacific (DEVCOM Indo-Pacific), under grant number N62909-23-1-2018 for Huynh Thi Thanh Binh, Ban Ha Bang, Le Minh Tu, Phan Duc Hung and Nguyen Thai Ha. The views and conclusions contained herein are those of the authors only and should not be interpreted as representing those of the U.S. Government.

## Declarations

- Conflict of interest:  
The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.
- Authors' contributions:  
Do Tuan Anh: Methodology, Algorithms, Coding, Writing the manuscript.  
Ha-Bang Ban: Methodology, Algorithms, Coding, Writing the manuscript.  
Huynh Thi Thanh Binh: Methodology, Algorithms, Writing the manuscript.  
Minh Tu Le: Methodology, Algorithms, Coding, Writing the manuscript.  
Phan Duc Hung: Coding, Writing the manuscript.  
Nguyen Thai Ha: Algorithms, Writing the manuscript.

## References

- [1] Chisman, J.A.: The clustered traveling salesman problem. *Computers & Operations Research* **2**(2), 115–119 (1975)
- [2] Lin, C.-W., Wu, B.Y.: On the minimum routing cost clustered tree problem. *Journal of Combinatorial Optimization* **33**(3), 1106–1121 (2017)
- [3] D'Emidio, M., Forlizzi, L., Frigioni, D., Leucci, S., Proietti, G.: On the clustered shortest-path tree problem. In: *ICTCS*, pp. 263–268 (2016)

- [4] Seah, W., Er, I.: Distributed steiner-like multicast path setup for mesh-based multicast routing in ad hoc networks **2**, 192–197 (2006). IEEE
- [5] Li, D., Jia, X., Liu, H.: Energy efficient broadcast routing in static ad hoc wireless networks. IEEE Transactions on Mobile Computing **3**(2), 144–151 (2004)
- [6] Cao, J., Guo, C., Lu, G., Xiong, Y., Zheng, Y., Zhang, Y., Zhu, Y., Chen, C., Tian, Y.: Datacast: A scalable and efficient reliable group data delivery service for data centers. IEEE Journal on Selected Areas in Communications **31**(12), 2632–2645 (2013)
- [7] Ding, C., Cheng, Y., He, M.: Two-level genetic algorithm for clustered traveling salesman problem with application in large-scale tsp. Tsinghua Science & Technology **12**(4), 459–465 (2007)
- [8] Potvin, J.-Y., Guertin, F.: The clustered traveling salesman problem: A genetic approach. In: Meta-Heuristics, pp. 619–631. Springer, ??? (1996)
- [9] Laporte, G., Potvin, J.-Y., Quilleret, F.: A tabu search heuristic using genetic diversification for the clustered traveling salesman problem. Journal of Heuristics **2**(3), 187–200 (1997)
- [10] Thang, T.B., Long, N.B., Hoang, N.V., Binh, H.T.T.: Adaptive knowledge transfer in multifactorial evolutionary algorithm for the clustered minimum routing cost problem. Applied Soft Computing **105**, 107253 (2021)
- [11] Lin, C.-W., Wu, B.Y.: A 2-approximation algorithm for the clustered minimum routing cost tree problem. In: Intelligent Systems and Applications, pp. 3–10. IOS Press, ??? (2015)
- [12] Garey, M.R., Graham, R.L., Johnson, D.S.: The complexity of computing steiner minimal trees. SIAM journal on applied mathematics **32**(4), 835–859 (1977)
- [13] Wu, B.Y., Lin, C.-W.: On the clustered steiner tree problem. Journal of Combinatorial Optimization **30**(2), 370–386 (2015)
- [14] Chen, Y.H.: The clustered selected-internal steiner tree problem. International Journal of Foundations of Computer Science **33**(01), 55–66 (2022)
- [15] Prim, R.C.: Shortest connection networks and some generalizations. The Bell System Technical Journal **36**(6), 1389–1401 (1957)

- [16] Kruskal, J.B.: On the shortest spanning subtree of a graph and the traveling salesman problem. *Proceedings of the American Mathematical society* **7**(1), 48–50 (1956)
- [17] Chen, L., Abdellatif, S., Gayraud, T., Berthou, P.: A steiner tree based approach for the efficient support of multipoint communications in a multi-domain context. In: *2017 IEEE Symposium on Computers and Communications (ISCC)*, pp. 316–321 (2017). IEEE
- [18] Do, T.A., Ban, H.-B., Huynh, T.T.B., Le, M.T., Nguyen, B.L.: Genetic algorithm based approach to solve the clustered steiner tree problem. *Evolutionary Intelligence*, 1–20 (2023)
- [19] Eshelman, L.J., Schaffer, J.D.: Real-coded genetic algorithms and interval-schemata **2**, 187–202 (1993)
- [20] Deb, K., Agrawal, S.: A niched-penalty approach for constraint handling in genetic algorithms. In: *Artificial Neural Nets and Genetic Algorithms*, pp. 235–243 (1999). Springer
- [21] Deb, K., Deb, D., *et al.*: Analysing mutation schemes for real-parameter genetic algorithms. *Int. J. Artif. Intell. Soft Comput.* **4**(1), 1–28 (2014)
- [22] Derrac, J., García, S., Molina, D., Herrera, F.: A practical tutorial on the use of nonparametric statistical tests as a methodology for comparing evolutionary and swarm intelligence algorithms. *Swarm and Evolutionary Computation* **1**(1), 3–18 (2011)
- [23] Gupta, A., Ong, Y.-S., Feng, L.: Multifactorial evolution: toward evolutionary multitasking. *IEEE Transactions on Evolutionary Computation* **20**(3), 343–357 (2015)